Rapport technique

VolyLegume — MaraîchèrePro

Application web de gestion d'exploitations maraîchères

Projet fil rouge — BUT RGI / L3 SIO — ESMIA

Version 1.0 — 23 octobre 2026

Résumé

VolyLegume est une application web de gestion d'exploitations maraîchères conçue pour le contexte malgache. Elle permet à une exploitation de piloter ses parcelles, ses cultures, ses campagnes, son calendrier cultural, ses activités, ses intrants et stocks, ses traitements phytosanitaires, ses récoltes et ses finances simplifiées. L'application est développée avec Laravel 13 et PHP 8.3, rendue côté serveur avec Blade et Tailwind CSS, et exposée partiellement via une API REST sécurisée par Sanctum.

Le présent rapport documente l'architecture, les choix techniques, les règles métier, le dispositif de sécurité, le journal d'audit, les tests et les conditions d'exploitation de la version livrée. Il met également en évidence la conformité au cahier des charges, les écarts résiduels et les perspectives d'évolution.

La version décrite est couverte par **93 tests automatisés et 284 assertions**, tous réussis. Elle comprend un contrôle d'accès centralisé par Policies Laravel, un journal d'audit transversal, un panneau d'administration de plateforme, un module de calendrier cultural et un module de fiches de culture.

Sommaire

1. Introduction et contexte
2. Besoins et conformité au cahier des charges
3. Architecture logicielle
4. Modèle de données
5. Sécurité et contrôle d'accès
6. Fonctionnalités métier
7. Journalisation et audit
8. API REST
9. Tests et qualité logicielle
10. Déploiement et exploitation
11. Bilan et perspectives
12. Annexes

---

1. Introduction et contexte

1.1 Contexte du projet

Le projet VolyLegume s'inscrit dans le cadre d'un projet fil rouge académique. Il répond à un besoin concret : les exploitations maraîchères de petite et moyenne taille disposent souvent de peu d'outils numériques adaptés à leur réalité quotidienne. Les registres papier, les tableurs et les échanges informels rendent difficile le suivi des parcelles, des intrants, des traitements et de la rentabilité.

L'application vise à fournir un outil simple, cohérent et traçable, utilisable par plusieurs personnes au sein d'une même exploitation. Elle doit également garantir qu'une exploitation ne puisse jamais accéder aux données d'une autre, même en manipulant les URL ou les identifiants.

Le contexte est celui de Madagascar, avec une devise unique en ariary (MGA), une localisation française et un fuseau horaire `Africa/Antananarivo`. Les données de démonstration sont fictives et ne constituent en aucun cas des prescriptions agronomiques.

1.2 Objectifs

Les objectifs poursuivis sont les suivants :

- centraliser les données de production d'une exploitation maraîchère ;
- suivre le cycle cultural, depuis la préparation du sol jusqu'à la récolte ;
- sécuriser les opérations sensibles, notamment les traitements phytosanitaires et les récoltes ;
- fournir des indicateurs de pilotage simples : marge, rendement, taux de réalisation, alertes de stock ;
- tracer les opérations pour faciliter le contrôle, la responsabilisation et la démonstration ;
- exposer une API REST permettant des intégrations futures ;
- garantir l'isolation des données entre exploitations ;
- fournir une base technique maintenable, testée et documentée.

1.3 Périmètre

Le périmètre fonctionnel couvre :

- la gestion des comptes et des rattachements à une ou plusieurs exploitations ;
- les rôles d'exploitation : administrateur, responsable, agent ;
- un rôle global d'administrateur de plateforme ;
- les parcelles et les cultures ;
- les campagnes culturales et le calendrier cultural ;
- les activités de suivi ;
- les intrants, les mouvements de stock et les alertes de seuil ;
- les traitements phytosanitaires et le délai avant récolte ;
- les récoltes ;
- les dépenses, recettes et indicateurs financiers simplifiés ;
- les notifications internes ;
- le journal d'audit ;
- l'API REST de consultation des campagnes et de saisie des activités.

Le périmètre exclut notamment :

- la comptabilité générale ou analytique complète ;
- la gestion de la paie ;
- la traçabilité réglementaire officielle ;
- la facturation client ;
- la gestion multi-devises ;
- les applications mobiles natives ;
- l'envoi de courriels transactionnels en production ;
- l'intégration avec des capteurs ou des stations météo.

1.4 Méthode de travail

La démarche adoptée est itérative. Elle a consisté à :

1. analyser le cahier des charges et le code existant ;
2. identifier les écarts entre les exigences et l'implémentation ;
3. renforcer les fondations techniques : Policies, audit, administration ;
4. compléter les tests automatisés ;
5. documenter l'architecture et les règles métier ;
6. vérifier la conformité par une matrice de couverture ;
7. formuler les limites et les perspectives.

Cette méthode a permis de traiter d'abord les risques structurants : autorisations, isolation multi-exploitations, traçabilité et administration. Les fonctionnalités métier déjà présentes ont été conservées et sécurisées par les Policies.

1.5 Public visé

Ce rapport s'adresse :

- à l'équipe pédagogique, pour l'évaluation technique et fonctionnelle ;
- aux développeurs qui reprendront le projet ;
- aux exploitants et responsables qui souhaitent comprendre le fonctionnement ;
- aux administrateurs chargés du déploiement.

---

2. Besoins et conformité au cahier des charges

2.1 Acteurs et rôles

L'application distingue deux niveaux de rôles.

2.1.1 Rôles d'exploitation

Le rôle est porté par le rattachement `farm_user.role`. Un utilisateur peut avoir des rôles différents selon l'exploitation.

Rôle	Libellé	Gestion des référentiels	Gestion des finances	Gestion de l'équipe	Saisie des activités
`admin`	Administrateur d'exploitation	Oui	Oui	Oui	Oui
`responsable`	Responsable / chef d'exploitation	Oui	Oui	Oui	Oui
`agent`	Agent / technicien	Non	Non	Non	Oui

Les capacités sont centralisées dans l'énumération `FarmRole`, ce qui évite de disperser des conditions dans les contrôleurs et les vues.

2.1.2 Administrateur de plateforme

Un utilisateur peut porter le booléen `is_platform_admin`. Ce rôle est transversal : il ne remplace pas les rôles d'exploitation, mais il autorise l'accès au panneau `/admin`. Il permet de :

- consulter les indicateurs globaux ;
- gérer les comptes ;
- activer ou désactiver un compte ;
- attribuer ou retirer le rôle de plateforme ;
- gérer les rattachements aux exploitations ;
- superviser les exploitations ;
- consulter le journal d'audit global.

Ce rôle répond au besoin d'administration qui n'était pas couvert par le rôle `admin` d'exploitation.

2.2 Besoins fonctionnels

Le tableau suivant synthétise les besoins fonctionnels et leur mise en œuvre.

Réf.	Besoin	Implémentation	État
BF01	Authentification	Session web + Sanctum pour l'API	Couvert
BF02	Inscription	`RegisterController`	Couvert
BF03	Déconnexion	`LogoutController`	Couvert
BF04	Multi-exploitations	`FarmContext`, `farm_user`	Couvert
BF05	Rôles et permissions	Enums + Policies	Couvert
BF06	Gestion des parcelles	CRUD `ProductionUnit`	Couvert
BF07	Gestion des cultures	CRUD `Crop` + catégories	Couvert
BF08	Fiches de culture	`CropGuide`, `CropGuideService`	Couvert
BF09	Calendrier cultural	`CalendarStep`, `CropCalendarService`	Couvert
BF10	Campagnes	CRUD `Campaign`	Couvert
BF11	Activités	CRUD et réalisation	Couvert
BF12	Intrants	CRUD `Input`	Couvert
BF13	Stock	`StockService`, mouvements	Couvert
BF14	Alertes de stock	Événement + notification + tableau de bord	Couvert
BF15	Traitements phytosanitaires	`TreatmentService`	Couvert
BF16	Délai avant récolte	Borne calculée côté serveur	Couvert
BF17	Récoltes	`HarvestService`	Couvert
BF18	Dépenses et recettes	CRUD + policies finances	Couvert

Réf.	Besoin	Implémentation	État
BF19	Tableau de bord	`DashboardService` + cache	Couvert
BF20	Notifications	Canal `farm` + base de données	Couvert
BF21	Journal d'audit	`AuditLog`, `AuditLogger`, `Auditable`	Couvert
BF22	Administration plateforme	`/admin` + policies	Couvert
BF23	API REST	Sanctum + ressources	Couvert
BF24	Recherche et pagination	Filtres et `paginate`	Couvert
BF25	File d'attente	`NotifyStockCritical`	Couvert

2.3 Règles de gestion principales

Les règles de gestion structurantes sont les suivantes.

RG01 — Isolation par exploitation

Toute ressource métier appartient à une exploitation. Le contexte actif est résolu côté serveur à partir des rattachements de l'utilisateur connecté. Aucun identifiant d'exploitation fourni par le client n'est accepté pour les routes web.

RG02 — Rôles

Un agent peut saisir des activités et consulter les données de son exploitation, mais ne peut pas gérer les référentiels, les finances, l'équipe ni les ajustements de stock. Un responsable ou un administrateur d'exploitation peut gérer ces éléments.

RG03 — Stock non négatif

Toute variation de stock passe par `StockService`. Le service verrouille la ligne d'intrant, calcule le solde après opération et refuse toute valeur négative. Le solde `inputs.quantity` reste cohérent avec la somme des mouvements.

RG04 — Traitement atomique

L'enregistrement d'un traitement crée dans une même transaction :

- une activité de type traitement ;
- un mouvement de stock de consommation ;
- une ligne de traitement phytosanitaire.

Si une étape échoue, l'ensemble est annulé.

RG05 — Délai avant récolte

La borne minimale de récolte est calculée côté serveur : `applied_at + pre_harvest_days × 24 heures`. La valeur est figée dans `minimum_harvest_at`. Une récolte antérieure à la borne la plus tardive applicable est refusée.

RG06 — Calendrier idempotent

La génération du calendrier ne duplique pas les activités existantes. Les activités réalisées ou annulées sont préservées ; seules les activités planifiées sont mises à jour.

RG07 — Traçabilité

Les créations, modifications et suppressions des modèles sensibles sont journalisées. Les connexions, les refus d'accès et les ressources masquées par l'isolation sont également tracés.

RG08 — Devise et unités

La devise est fixée à MGA. Les quantités de production et de stock sont exprimées en kilogrammes pour les récoltes et selon l'unité de base de l'intrant pour les stocks.

RG09 — Validation serveur

Les règles de validation sont appliquées côté serveur. Les contrôles côté client ne sont qu'une aide à la saisie.

RG10 — Administration séparée

L'administration de plateforme est distincte de l'administration d'exploitation. Un administrateur d'exploitation ne peut pas accéder au panneau global s'il n'est pas `is_platform_admin`.

2.4 Besoins non fonctionnels

Réf.	Besoin	Réponse
BNF01	Sécurité	Policies, CSRF, validation, hachage Argon2id, Sanctum
BNF02	Isolation	Contexte serveur + `BelongsToFarm`
BNF03	Traçabilité	Journal d'audit transversal
BNF04	Maintenabilité	Architecture MVC + services + Policies
BNF05	Testabilité	PHPUnit, SQLite en mémoire, 93 tests
BNF06	Performance	Cache tableau de bord, index SQL, pagination
BNF07	Robustesse	Transactions, verrous pessimistes, contraintes SQL
BNF08	Portabilité	Docker, PostgreSQL, configuration `.env`
BNF09	Accessibilité	Interface Blade responsive, libellés français
BNF10	Évolutivité	API REST, services découplés

2.5 Matrice de conformité

Exigence du cahier des charges	Réalisation	Preuve
Rapport technique 15–25 pages	Ce document	Sections 1 à 12
Policies Laravel	14 Policies + trait commun	`app/Policies`
Journalisation	`audit_logs`, `AuditLogger`, `Auditable`	`app/Support`, `app/Models/Concerns`
Administration globale	Panneau `/admin`	`app/Http/Controllers/Admin`
Tests	93 tests, 284 assertions	`php artisan test`
Base de données 8–15 tables	18 tables métier et techniques	`database/migrations`
Factories et seeders	Factories par modèle + `DatabaseSeeder`	`database/factories`, `database/seeders`
Module personnel	Calendrier cultural + fiches de culture	`CropCalendarService`, `CropGuideService`
Isolation multi-exploitations	Contexte serveur + route binding	`FarmContext`, `BelongsToFarm`
API REST	Sanctum + endpoints	`routes/api.php`

2.6 Écarts résiduels

Les écarts identifiés sont les suivants :

- l'historique Git reste limité et doit être enrichi par des commits explicites ;
- la couverture de tests peut encore être étendue aux cas limites de l'API ;
- la rétention du journal d'audit n'est pas automatisée ;
- l'envoi de courriels réels n'est pas configuré ;
- la documentation utilisateur reste synthétique ;
- la supervision de production (Sentry, métriques) n'est pas installée.

Ces écarts ne remettent pas en cause les fonctionnalités principales, mais ils constituent des axes d'amélioration.

---

3. Architecture logicielle

3.1 Vue d'ensemble

L'application suit l'architecture MVC de Laravel, complétée par des services métier et des Policies.

```
Navigateur / client API
        |
        v
+------------------------------+
| Routes web et API            |
| Middleware (auth, farm, admin)|
+------------------------------+
        |
        v
+------------------------------+
| Contrôleurs                  |
| - validation                 |
| - autorisation (Policies)    |
| - orchestration              |
+------------------------------+
        |
        v
+------------------------------+
| Services métier              |
| Stock, Traitement, Récolte,  |
| Calendrier, Dashboard        |
+------------------------------+
        |
        v
+------------------------------+
| Modèles Eloquent             |
| Relations, scopes, casts     |
+------------------------------+
        |
        v
+------------------------------+
| PostgreSQL / SQLite (tests)  |
+------------------------------+
```

Le journal d'audit est un composant transversal : il est déclenché par les événements Eloquent et par les services, et écrit dans la table `audit_logs`.

3.2 Organisation des dossiers

```
app/
  Console/Commands/       Commande de génération de données
  Enums/                  Énumérations métier
  Events/                 Événements de domaine
  Exceptions/             Exceptions métier
  Http/
    Controllers/          Contrôleurs web
    Controllers/Admin/    Administration plateforme
    Controllers/Api/      API REST
    Middleware/           Contexte, isolation, administration
    Requests/             Validation des activités
    Resources/            Sérialisation API
  Jobs/                   Traitements différés
  Listeners/              Réactions aux événements
  Models/                 Modèles Eloquent
  Models/Concerns/        Traits Auditable et BelongsToFarm
  Notifications/          Notifications et canal farm
  Policies/               Autorisations
  Providers/              Configuration applicative
  Services/               Logique métier
  Support/                Contexte, cache, audit
database/
  factories/              Fabriques de données
  migrations/             Schéma
  seeders/                Données de démonstration
resources/views/          Interface Blade
routes/                   web.php, api.php, console.php
tests/                    Tests unitaires et fonctionnels
```

3.3 Rôle des couches

3.3.1 Contrôleurs

Les contrôleurs restent minces. Ils :

- valident les entrées ;
- autorisent l'action via `Gate::authorize` ou les Policies ;
- délèguent les opérations complexes aux services ;
- retournent une vue ou une redirection.

Les contrôleurs d'administration vérifient le rôle de plateforme dans leur constructeur via `Gate::authorize('manage', User::class)`.

3.3.2 Services

Les services portent les règles métier qui ne doivent pas être dupliquées :

Service	Responsabilité
`StockService`	Mouvements, verrouillage, solde, seuil critique
`TreatmentService`	Traitement atomique et borne de récolte
`HarvestService`	Contrôle du délai et enregistrement
`CropCalendarService`	Prévisualisation, génération, replanification
`CropGuideService`	Fiche de culture et étapes
`DashboardService`	Agrégats et cache

3.3.3 Modèles

Les modèles Eloquent portent :

- les relations ;
- les casts ;
- les scopes (`forFarm` , `overdue` , etc.) ;
- les traits transversaux (`Auditable` , `BelongsToFarm`) ;
- les contraintes de cohérence applicatives.

3.3.4 Policies

Les Policies centralisent les décisions d'autorisation. Le trait `ChecksFarmRole` fournit les primitives :

- rôle courant ;
- appartenance à l'exploitation ;
- capacité de gestion des référentiels ;
- capacité de gestion des finances ;
- appartenance d'un modèle à l'exploitation active.

3.4 Flux d'une requête web

Prenons l'exemple de la création d'une dépense.

1. Le navigateur envoie `POST /finances/depenses` .
2. Le middleware `auth` vérifie la session.
3. `ResolveFarmContext` lit les rattachements actifs et choisit l'exploitation active.
4. `RequireActiveFarm` vérifie qu'une exploitation est disponible.
5. Le contrôleur valide la requête.
6. `ExpensePolicy::create` vérifie que l'utilisateur peut gérer les finances.
7. `Expense::create` insère la ligne.
8. Le trait `Auditable` déclenche `AuditLogger` .
9. `DashboardCache` est invalidé.
10. Une redirection est renvoyée avec un message de succès.

3.5 Flux d'une requête API

1. Le client envoie un token Bearer.

2. `auth:sanctum` authentifie l'utilisateur.
3. `ResolveApiFarmContext` lit l'en-tête `X-Farm-Id` et vérifie le rattachement.
4. `RequireActiveFarm` refuse si aucun contexte.
5. La ressource est chargée avec `resolveRouteBinding`, qui vérifie l'appartenance à l'exploitation.
6. Le contrôleur retourne une ressource JSON.

3.6 Choix techniques

Laravel 13

Laravel fournit l'essentiel : routage, ORM, validation, sessions, files d'attente, notifications, tests. La version 13 impose PHP 8.3 et modernise la configuration via `bootstrap/app.php`.

Blade et Tailwind

Blade permet un rendu serveur rapide, sans complexité d'application monopage. Tailwind CSS fournit une interface responsive et cohérente.

PostgreSQL

PostgreSQL est utilisé en développement et en production locale. Il permet les contraintes `CHECK`, les clés étrangères composites et les verrous pessimistes. SQLite est utilisé uniquement pour les tests.

Sanctum

Sanctum fournit une authentification par token simple pour l'API, sans OAuth complet.

PHPUnit

PHPUnit 12 est le framework de test natif de Laravel. Les tests utilisent SQLite en mémoire pour être rapides et isolés.

---

4. Modèle de données

4.1 Vue relationnelle

```
users 1 ──── n farm_user n ──── 1 farms
farms 1 ──── n production_units
farms 1 ──── n crops
farms 1 ──── n inputs
farms 1 ──── n campaigns
production_units 1 ──── n campaigns
crops 1 ──── n campaigns
crops 1 ──── 1 crop_guides
crop_categories 1 ──── n crops
crops 1 ──── n calendar_steps
campaigns 1 ──── n activities
campaigns 1 ──── n harvests
campaigns 1 ──── n treatments
campaigns 1 ──── n expenses
campaigns 1 ──── n revenues
inputs 1 ──── n stock_movements
activities 1 ──── 0..1 treatments
stock_movements 1 ──── 0..1 treatments
harvests 1 ──── n revenues
users 1 ──── n audit_logs
farms 1 ──── n audit_logs
```

4.2 Tables principales

`users`

Compte utilisateur. Champs notables :

- `name`, `email`, `password` ;
- `is_active` : un compte désactivé ne peut plus se connecter ;

- `is_platform_admin` : rôle global d'administration ;
- `email_verified_at`, `remember_token`.

`farms`

Exploitation agricole.

- `name`, `location`, `area_m2` ;
- `manager_id` : responsable principal ;
- `status` : `draft`, `active`, `inactive` ;
- `currency` : `MGA`.

`farm_user`

Rattachement d'un utilisateur à une exploitation.

- `farm_id`, `user_id`, `role` ;
- `is_active` ;
- unicité `(farm_id, user_id)`.

`production_units`

Parcelle ou unité de production.

- `code`, `name`, `area_m2`, `location` ;
- `type` : `plot` ;
- `status` : `available`, `occupied`, `resting`, `inactive` ;
- unicité `(farm_id, code)`.

`crops` et `crop_categories`

Référentiel des cultures.

- `name`, `variety`, `production_unit`, `estimated_cycle_days` ;
- `category_id` ;
- `is_active` ;
- unicité `(farm_id, name, variety)`.

`crop_guides`

Fiche de culture générée à partir de la catégorie et de l'espèce.

- contenu textuel : présentation, variétés, climat, sol, préparation, semis, irrigation, fertilisation, entretien, ravageurs, récolte, rendement, conservation, sources ;
- étapes communes, étapes de catégorie et étapes spécifiques.

`calendar_steps`

Étapes du calendrier cultural d'une culture.

- `crop_id`, `position`, `name`, `description` ;
- `offset_days` ;
- `activity_type`, `is_active`.

`campaigns`

Campagne de production.

- `production_unit_id`, `crop_id` ;
- `name`, `start_date`, `expected_end_date`, `actual_end_date` ;
- `reference_type` : `sowing`, `planting` ;
- `cultivated_area_m2`, `status`, `notes`.

`activities`

Activité planifiée ou réalisée.

- `campaign_id`, `calendar_step_id` éventuel ;
- `type`, `title`, `description` ;
- `planned_date`, `performed_at`, `status` ;
- `assigned_to`, `created_by`, `performed_by` ;

- `estimated_cost`.

`inputs`

Intrant ou produit.

- `code`, `name`, `category` ;
- `base_unit`, `quantity`, `minimum_stock` ;
- `default_pre_harvest_days`, `delay_source` ;
- `critical_notified_at`, `is_active`.

`stock_movements`

Mouvement de stock.

- `input_id`, `campaign_id`, `activity_id` ;
- `type` : `initial`, `entry`, `exit`, `consumption`, `adjustment` ;
- `quantity_delta`, `occurred_at`, `reason` ;
- `created_by`, `corrects_movement_id`.

`phytosanitary_treatments`

Traitement phytosanitaire.

- `campaign_id`, `input_id`, `activity_id`, `stock_movement_id` ;
- `applied_at`, `dose_value`, `dose_unit` ;
- `pre_harvest_days`, `delay_source_snapshot` ;
- `minimum_harvest_at`, `applied_by`, `notes`.

`harvests`

Récolte.

- `campaign_id`, `harvested_at`, `quantity`, `unit` ;
- `quality`, `recorded_by`, `notes`.

`expenses` et **`revenues`**

Finances simplifiées.

- `campaign_id` éventuel ;
- `category`, `description`, `amount`, `date` ;
- `reference`, `recorded_by` ;
- pour les recettes : `harvest_id` éventuel.

`audit_logs`

Journal transversal.

- `farm_id`, `user_id` ;
- `action`, `description` ;
- `auditable_type`, `auditable_id` ;
- `old_values`, `new_values`, `context` ;
- `ip_address`, `user_agent`, `created_at`.

4.3 Contraintes d'intégrité

Les migrations ajoutent des contraintes SQL lorsque le driver est PostgreSQL :

- montants strictement positifs ;
- surfaces strictement positives ;
- statuts limités à une liste ;
- types limités à une liste ;
- signe des mouvements de stock cohérent avec le type ;
- borne de récolte postérieure ou égale à l'application du traitement ;
- unité de récolte limitée à `kg` ;
- devise limitée à `MGA`.

Ces contraintes complètent la validation applicative et protègent contre les écritures incohérentes, y compris en cas de script ou d'import.

4.4 Index et performance

Les index couvrent les requêtes fréquentes :

- `(farm_id, status)` sur les parcelles ;
- `(farm_id, planned_date)` sur les activités ;
- `(farm_id, input_id, occurred_at)` sur les mouvements ;
- `(farm_id, campaign_id, applied_at)` sur les traitements ;
- `(farm_id, campaign_id, harvested_at)` sur les récoltes ;
- `(farm_id, expense_date)` et `(farm_id, revenue_date)` sur les finances ;
- `(farm_id, created_at)` et `(user_id, created_at)` sur l'audit.

Les listes sont paginées par 20 éléments. Le tableau de bord est mis en cache dix minutes.

---

5. Sécurité et contrôle d'accès

5.1 Authentification web

L'authentification web repose sur les sessions Laravel. Les routes protégées utilisent le middleware `auth`. La déconnexion invalide la session et régénère le jeton CSRF.

Les mots de passe sont hachés avec Argon2id lorsque la configuration le permet. Les anciens hachages bcrypt restent vérifiables et sont rehachés à la première connexion réussie. La configuration recommande :

```
HASH_DRIVER=argon2id
ARGON_MEMORY=65536
ARGON_TIME=4
ARGON_THREADS=1
```

5.2 Authentification API

L'API utilise Sanctum. Le client obtient un token via `POST /api/login`, puis l'envoie dans l'en-tête `Authorization: Bearer <token>`. La déconnexion révoque le token courant.

Le token ne porte pas de contexte d'exploitation : celui-ci est fourni par l'en-tête `X-Farm-Id` et vérifié côté serveur. Un utilisateur ne peut choisir qu'une exploitation à laquelle il est rattaché et actif.

5.3 Contexte d'exploitation

Le contexte est un service `FarmContext` de portée requête. Il contient :

- l'exploitation active ;
- le rôle de l'utilisateur dans cette exploitation.

Pour le web, `ResolveFarmContext` :

1. lit les rattachements actifs de l'utilisateur ;
2. privilégie l'exploitation mémorisée en session ;
3. sinon prend la première exploitation disponible ;
4. met à jour la session ;
5. alimente `FarmContext`.

Pour l'API, `ResolveApiFarmContext` lit `X-Farm-Id` et vérifie le rattachement.

Aucun identifiant d'exploitation n'est accepté directement dans les URL web pour choisir le contexte. Les routes d'administration sont l'exception contrôlée : elles opèrent explicitement sur toutes les exploitations, mais elles sont réservées aux administrateurs de plateforme.

5.4 Policies

Les Policies sont déclarées dans `app/Policies` et découvertes automatiquement par Laravel selon la convention de nommage.

Modèle	Policy	Règles principales
`Farm`	`FarmPolicy`	membre pour voir, responsable/admin pour modifier et gérer l'équipe
`ProductionUnit`	`ProductionUnitPolicy`	membre pour voir, responsable/admin pour créer/modifier/supprimer
`Crop`	`CropPolicy`	idem
`CalendarStep`	`CalendarStepPolicy`	idem
`Campaign`	`CampaignPolicy`	idem
`Activity`	`ActivityPolicy`	membre pour voir et créer, responsable/admin pour modifier/supprimer
`Input`	`InputPolicy`	membre pour voir, responsable/admin pour créer/modifier/supprimer/adjuster
`StockMovement`	`StockMovementPolicy`	membre pour voir et créer, responsable/admin pour ajuster
`PhytosanitaryTreatment`	`TreatmentPolicy`	membre pour voir, responsable/admin pour créer
`Harvest`	`HarvestPolicy`	membre pour voir et créer, responsable/admin pour supprimer
`Expense`	`ExpensePolicy`	membre pour voir, responsable/admin pour gérer
`Revenue`	`RevenuePolicy`	membre pour voir, responsable/admin pour gérer
`AuditLog`	`AuditLogPolicy`	responsable/admin pour consulter, dans l'exploitation active
`User`	`UserPolicy`	administrateur de plateforme uniquement

Le trait `ChecksFarmRole` évite la duplication :

```
protected function currentRole(): ?FarmRole
{
    return app(FarmContext::class)->role();
}

protected function canManageFinances(): bool
{
    return $this->currentRole()?->canManageFinances() ?? false;
}

protected function belongsToCurrentFarm(Model $model): bool
{
    return (int) $model->getAttribute('farm_id')
        === (int) app(FarmContext::class)->id();
}
```

5.5 Administrateur de plateforme

Le rôle de plateforme est accordé par `is_platform_admin`. Un `Gate::before` global autorise toutes les abilities pour cet utilisateur :

```
Gate::before(function (User $user): ?bool {
    return $user->isPlatformAdmin() ? true : null;
});
```

Ce mécanisme est volontairement simple. Il est complété par des contrôles métier dans les contrôleurs d'administration :

- interdiction de désactiver son propre compte ;
- interdiction de retirer son propre rôle de plateforme ;
- interdiction de retirer le dernier administrateur de plateforme ;
- interdiction de retirer le rôle Responsable au responsable principal d'une exploitation.

5.6 Isolation multi-exploitations

L'isolation repose sur trois niveaux.

Niveau 1 — Contexte

Le contexte est résolu côté serveur. Un utilisateur ne peut activer qu'une exploitation à laquelle il est rattaché et actif.

Niveau 2 — Requêtes

Les modèles métier utilisent le trait `BelongsToFarm` :

- le scope `forFarm` restreint les listes ;
- `resolveRouteBinding` renvoie un 404 si la ressource appartient à une autre exploitation.

Niveau 3 — Contraintes

Les clés étrangères composites lient les entités à la même exploitation. Par exemple, une campagne ne peut référencer qu'une parcelle et une culture de la même exploitation.

Cette défense en profondeur empêche les fuites par URL, par identifiant ou par requête mal formée.

5.7 Validation et protections web

- Validation systématique des entrées via `$request->validate` ou `FormRequest`.
- Protection CSRF sur les formulaires web.
- Échappement Blade par défaut contre XSS.
- Assignation de masse limitée par `$fillable`.
- Requêtes SQL paramétrées par Eloquent.
- Cookies de session chiffrés.
- Limitation de débit sur l'API (`throttle:api` , 60 requêtes/minute).
- Messages d'erreur génériques pour les 403 et 404.

5.8 Journalisation des refus

Les refus d'accès sont journalisés par `DeniedAccessLogger`, appelé depuis le gestionnaire d'exceptions. Il couvre :

- `AuthorizationException` (403) ;
- `AccessDeniedHttpException` (403) ;
- `ModelNotFoundException` (404 d'isolation).

Le journal enregistre la méthode HTTP, le chemin, la route, le statut et l'utilisateur. Il ne journalise pas les requêtes anonymes.

---

6. Fonctionnalités métier

6.1 Parcelles et cultures

Les parcelles sont les unités physiques de production. Elles portent un code unique par exploitation, une surface, un emplacement et un statut.

Les cultures sont des référentiels propres à l'exploitation. Chaque culture peut être associée à une catégorie (légumes-feuilles, légumes-fruits, racines, etc.) et à une fiche de culture.

La fiche de culture est générée à partir de :

- étapes communes à toutes les cultures ;
- étapes spécifiques à la catégorie ;
- points spécifiques connus pour l'espèce ;
- variété et durée de cycle renseignées.

Le contenu reste qualitatif : il ne remplace pas les recommandations locales.

6.2 Calendrier cultural

Le calendrier cultural est le module personnel du projet. Il permet de générer automatiquement les activités prévues d'une campagne à partir des étapes de la culture.

Fonctionnement :

1. chaque étape possède un décalage en jours (`offset_days`) ;
2. la date planifiée est calculée depuis `campaigns.start_date` ;
3. la génération est idempotente ;
4. les activités planifiées sont mises à jour ;
5. les activités réalisées ou annulées sont préservées ;
6. la replanification ne touche que les activités planifiées.

Le service retourne un bilan :

```
{ created: n, updated: n, preserved: n }
```

Cette conception évite les doublons et respecte le travail déjà réalisé sur le terrain.

6.3 Campagnes

Une campagne associe une parcelle, une culture, une date de début, une date de fin prévue, une surface cultivée et un statut.

Statuts :

- `planned` : planifiée ;
- `ongoing` : en cours ;
- `suspended` : suspendue ;
- `completed` : terminée ;
- `cancelled` : annulée.

Une campagne peut être associée à un type de référence : semis ou plantation.

6.4 Activités

Les activités couvrent :

- préparation du sol ;
- semis ;
- repiquage ;
- fertilisation ;
- traitement ;
- irrigation ;
- désherbage ;
- récolte ;
- observation ;
- entretien.

Une activité peut être planifiée, réalisée ou annulée. Elle peut être assignée à un utilisateur et exécutée par un autre. Les activités en retard sont détectées par le scope `overdue`.

6.5 Intrants et stocks

Les intrants sont catégorisés : semences, engrais, produits phytosanitaires, carburant, consommables. Chaque intrant possède :

- une unité de base ;
- une quantité courante ;
- un seuil minimal ;
- un délai avant récolte par défaut pour les phytosanitaires ;
- une source du délai.

Toute variation passe par `StockService` :

- `setInitial` : stock initial ;
- `receive` : entrée ;
- `consume` : consommation ;
- `release` : sortie ;
- `adjust` : ajustement signé.

Le service :

- arrondit à trois décimales ;
- refuse une variation nulle ;
- verrouille la ligne ;
- refuse un solde négatif ;
- met à jour `critical_notified_at` ;
- déclenche `StockCritical` au passage sous le seuil ;

- invalide le cache du tableau de bord.

6.6 Traitements phytosanitaires

Un traitement est enregistré via `TreatmentService`. Le service vérifie :

- la quantité positive ;
- le délai non négatif ;
- l'appartenance de l'intrant et de la campagne à la même exploitation ;
- le caractère utilisable du produit phytosanitaire.

Il crée ensuite, dans une transaction :

1. une activité réalisée ;
2. un mouvement de stock de consommation ;
3. le traitement avec sa borne de récolte.

La borne est figée dans `minimum_harvest_at`. La source du délai est snapshotée pour conserver la preuve au moment de l'application.

6.7 Récoltes

`HarvestService` vérifie la quantité et la borne de récolte. La borne applicable est la plus tardive des bornes des traitements appliqués au plus tard à la date de récolte.

Si la date de récolte est antérieure, une `HarvestTooEarlyException` est levée et la récolte est refusée.

Cette règle est bloquante côté serveur et ne peut pas être contournée par le client.

6.8 Finances

Les finances sont simplifiées :

- dépenses catégorisées ;
- recettes catégorisées ;
- rattachement optionnel à une campagne ;
- rattachement optionnel d'une recette à une récolte.

Les indicateurs du tableau de bord sont :

- total des dépenses ;
- total des recettes ;
- marge simplifiée ;
- campagnes actives ;
- nombre et surface des parcelles ;
- intrants sous seuil ;
- activités en retard ;
- taux de réalisation ;
- quantité récoltée ;
- rendement kg/ha ;
- activités récentes.

Le calcul est isolé par exploitation et mis en cache.

6.9 Notifications

Le projet utilise le système de notifications Laravel avec un canal personnalisé `farm`. Les notifications sont stockées en base et filtrées par exploitation.

L'événement `StockCritical` est traité par un listener qui envoie une notification. Le traitement peut être différé via la file d'attente.

6.10 Tableau de bord et cache

`DashboardService` calcule les agrégats à la demande et les met en cache dix minutes. La clé inclut :

- l'identifiant de l'exploitation ;
- une version de cache par exploitation ;

- un hash des filtres.

`DashboardCache::forget` incrémente la version, ce qui invalide toutes les combinaisons de filtres sans parcourir le cache.

---

7. Journalisation et audit

7.1 Objectifs

Le journal d'audit répond à plusieurs besoins :

- savoir qui a fait quoi et quand ;
- reconstituer une opération sensible ;
- détecter les tentatives d'accès non autorisées ;
- faciliter la démonstration et le contrôle ;
- conserver les valeurs avant/après pour les modifications.

7.2 Modèle `audit_logs`

Chaque ligne contient :

Champ	Rôle
`farm_id`	exploitation concernée, nullable
`user_id`	acteur, nullable
`action`	code stable, ex. `expense.created`
`description`	texte lisible
`auditable_type`	classe du sujet
`auditable_id`	identifiant du sujet
`old_values`	valeurs avant modification
`new_values`	valeurs après création/modification
`context`	méthode, chemin, route, statut
`ip_address`	adresse IP
`user_agent`	agent utilisateur
`created_at`	horodatage

Le modèle `AuditLog` fournit :

- les casts JSON ;
- les relations `farm`, `user`, `auditable` ;
- le scope `forFarm` ;
- le scope `search` ;
- un libellé d'action lisible.

7.3 Trait `Auditable`

Les modèles suivants utilisent `Auditable` :

- `Farm` ;
- `FarmUser` ;
- `ProductionUnit` ;
- `Crop` ;
- `CalendarStep` ;
- `Campaign` ;
- `Activity` ;
- `Input` ;
- `StockMovement` ;
- `PhytosanitaryTreatment` ;
- `Harvest` ;

- `Expense` ;
- `Revenue` .

Le trait écoute les événements Eloquent `created` , `updated` et `deleted` , puis appelle `AuditLogger::recordModelEvent` .

7.4 `AuditLogger`

`AuditLogger` centralise l'écriture. Il :

- déduit l'utilisateur depuis la requête ou `Auth` ;
- déduit l'exploitation depuis le sujet ou le contexte ;
- exclut les champs sensibles (`password` , `remember_token` , timestamps) ;
- limite la taille des chaînes ;
- convertit toute erreur d'écriture en avertissement de log applicatif.

Le journal ne doit jamais interrompre une opération métier.

7.5 Événements tracés

Catégorie	Exemples
Création	`farm.created` , `campaign.created` , `expense.created`
Modification	`crop.updated` , `input.updated`
Suppression	`harvest.deleted` , `activity.deleted`
Authentification	`auth.login` , `auth.login_failed` , `auth.api_login`
Refus	`access.denied`
Administration	`admin.user.created` , `admin.user.deactivated` , `admin.membership.updated` , `admin.farm.status`

7.6 Consultation

Deux écrans sont disponibles :

- `/journal` : journal de l'exploitation active, accessible aux responsables et administrateurs d'exploitation ;
- `/admin/journal` : journal transversal, accessible aux administrateurs de plateforme.

Les deux écrans permettent la recherche par action ou description, la pagination et l'affichage du contexte.

7.7 Limites et rétention

La version actuelle ne met pas en place de purge automatique. Une politique de rétention devra être définie selon les besoins :

- conservation longue pour les opérations financières et les traitements ;
- purge ou archivage des connexions et refus après une durée définie ;
- export périodique pour archivage.

Le journal ne stocke pas les valeurs binaires ni les pièces jointes. Il ne remplace pas un système de sauvegarde.

---

8. API REST

8.1 Principes

L'API est exposée sous `/api` . Elle utilise :

- Sanctum pour l'authentification ;
- `X-Farm-Id` pour le contexte ;
- des ressources JSON pour la sérialisation ;
- des codes HTTP standards ;
- une limitation de débit.

8.2 Endpoints

Méthode	Route	Description	Auth
POST	`/api/login`	Obtenir un token	Non
POST	`/api/logout`	Révoquer le token	Oui
GET	`/api/campaigns`	Lister les campagnes	Oui
GET	`/api/campaigns/{campaign}`	Détail d'une campagne	Oui
POST	`/api/activities`	Créer une activité	Oui

8.3 Contexte `X-Farm-Id`

Le client doit fournir l'exploitation cible. Le middleware vérifie :

- que l'utilisateur est authentifié ;
- que le rattachement existe ;
- que le rattachement est actif ;
- que l'exploitation existe.

Sinon, une réponse 403 est renvoyée.

8.4 Réponses et erreurs

Les réponses de liste incluent des en-têtes de pagination. Les erreurs suivent les codes HTTP :

- 200 : succès ;
- 201 : création ;
- 401 : non authentifié ;
- 403 : non autorisé ou contexte absent ;
- 404 : ressource introuvable ou hors exploitation ;
- 422 : validation échouée ;
- 429 : limite de débit atteinte.

8.5 Sérialisation

Les ressources `CampaignResource` et `ActivityResource` exposent uniquement les champs utiles. Elles évitent d'exposer les relations internes ou les données sensibles.

8.6 Évolutions possibles

- endpoints de consultation des parcelles et intrants ;
- endpoints de traitement et de récolte ;
- pagination par curseur ;
- versionnement `/api/v1` ;
- documentation OpenAPI ;
- webhooks.

---

9. Tests et qualité logicielle

9.1 Stratégie

La stratégie de test combine :

- tests unitaires pour les composants isolés ;
- tests fonctionnels pour les parcours HTTP ;
- tests d'intégration pour les services et la base ;
- tests d'isolation multi-exploitations ;
- tests d'autorisation ;
- tests d'audit ;
- tests d'administration.

Les tests utilisent `RefreshDatabase` et SQLite en mémoire. Chaque test est isolé.

9.2 Organisation

```
tests/
  Concerns/InteractsWithFarm.php
  Feature/
    AdminPanelTest.php
    ApiActivitiesTest.php
    ApiAuthTest.php
    ApiCampaignsTest.php
    AuditLogTest.php
    Auth/AuthenticationTest.php
    CalendarGenerationTest.php
    CalendarTest.php
    CropCatalogTest.php
    DashboardCacheTest.php
    DashboardCounterTest.php
    EnumOptionsRenderingTest.php
    FarmIsolationTest.php
    FinanceAuthorizationTest.php
    NotificationFarmScopeTest.php
    PasswordHashingTest.php
    PolicyAuthorizationTest.php
    ResourceCenterTest.php
    StockManagementTest.php
    TeamManagementTest.php
    TreatmentAndHarvestTest.php
  Unit/
    ExampleTest.php
```

9.3 Couverture fonctionnelle

Domaine	Tests représentatifs
Authentification	`AuthenticationTest`, `ApiAuthTest`
Rôles et Policies	`PolicyAuthorizationTest`, `FinanceAuthorizationTest`, `TeamManagementTest`
Isolation	`FarmIsolationTest`, `NotificationFarmScopeTest`
Stock	`StockManagementTest`
Traitement et récolte	`TreatmentAndHarvestTest`
Calendrier	`CalendarTest`, `CalendarGenerationTest`
Cultures et ressources	`CropCatalogTest`, `ResourceCenterTest`
Tableau de bord	`DashboardCounterTest`, `DashboardCacheTest`
Audit	`AuditLogTest`
Administration	`AdminPanelTest`
API	`ApiAuthTest`, `ApiCampaignsTest`, `ApiActivitiesTest`
Sécurité des mots de passe	`PasswordHashingTest`
Rendu des énumérations	`EnumOptionsRenderingTest`

9.4 Résultats

Commande exécutée :

```
php artisan test --compact
```

Résultat :

```
Tests:    93 passed (284 assertions)
Duration: 22.68s
```

Les tests couvrent notamment :

- l'accès interdit à un agent pour les référentiels et finances ;
- l'accès autorisé pour un responsable ;
- l'isolation des ressources par exploitation ;
- la journalisation des refus ;
- la journalisation des opérations métier ;
- l'administration plateforme ;
- la création de comptes et de rattachements ;
- le blocage des récoltes trop précoces ;
- le stock non négatif ;
- la génération idempotente du calendrier ;
- l'authentification API ;
- le hachage des mots de passe.

9.5 Formatage

Le projet utilise Laravel Pint :

```
vendor/bin/pint
```

Le formatage est vérifié avant livraison. Les règles suivent le style Laravel/PSR-12.

9.6 Limites des tests

- SQLite ne reproduit pas toutes les contraintes PostgreSQL, notamment certaines contraintes `CHECK`.
- Les tests ne couvrent pas encore tous les cas limites de concurrence.
- Les tests d'interface restent limités ; il n'y a pas de tests end-to-end navigateur.
- La file d'attente est testée de manière fonctionnelle, pas en conditions de charge.
- Les performances ne font pas l'objet de tests dédiés.

---

10. Déploiement et exploitation

10.1 Prérequis

- PHP 8.3 ou supérieur ;
- Composer ;
- Node.js et npm ;
- PostgreSQL 18 ou compatible ;
- extension PHP `pdo_pgsql` ;
- serveur web ou `php artisan serve` pour le développement.

10.2 Installation

```
composer install
cp .env.example .env
php artisan key:generate
npm install
npm run build
php artisan migrate --seed
php artisan serve
```

10.3 Docker

Le projet fournit un `docker-compose.yml` pour PostgreSQL. Le port exposé est `5433` afin d'éviter les conflits avec une instance locale.

```
docker compose up -d
```

La configuration `.env.example` pointe vers :

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5433
DB_DATABASE=volylegume
DB_USERNAME=volylegume
DB_PASSWORD=volylegume_secret
```

10.4 Variables d'environnement

Les variables critiques sont :

- `APP_ENV`, `APP_DEBUG`, `APP_KEY` ;
- `APP_URL`, `APP_TIMEZONE`, `APP_LOCALE` ;
- `DB_*` ;
- `SESSION_DRIVER`, `SESSION_LIFETIME` ;
- `CACHE_STORE`, `QUEUE_CONNECTION` ;
- `HASH_DRIVER`, `ARGON_*`, `BCRYPT_ROUNDS` ;
- `MAIL_*`.

En production :

- `APP_DEBUG=false` ;
- `APP_ENV=production` ;
- HTTPS obligatoire ;
- cookies sécurisés ;
- secrets stockés hors du dépôt ;
- sauvegardes régulières.

10.5 Migrations et seeders

```
php artisan migrate:fresh --seed
```

Le seeder crée :

- deux exploitations ;
- des parcelles, cultures, campagnes, activités ;
- des intrants avec alertes de stock ;
- des traitements et récoltes de démonstration ;
- des comptes de démonstration.

Comptes principaux :

Compte	Rôle	Exploitation
`responsable@volylegume.mg`	Responsable	Ferme Ambohipo
`agent+ambohipo@volylegume.mg`	Agent	Ferme Ambohipo
`admin@volylegume.mg`	Admin plateforme + Admin	Toutes
`responsable.itasy@volylegume.mg`	Responsable	Itasy
`agent+analavory@volylegume.mg`	Agent	Itasy

Mot de passe commun : `password`.

10.6 File d'attente

La file utilise la base de données :

```
php artisan queue:work
```

En production, un superviseur (systemd, Supervisor) doit maintenir le worker. Les notifications de stock critique peuvent être différées.

10.7 Cache et sessions

Le cache et les sessions utilisent la base de données. Les tables `cache`, `sessions` et `jobs` sont créées par les migrations.

10.8 Sauvegarde

La sauvegarde doit couvrir :

- la base PostgreSQL ;
- les fichiers de stockage ;
- les variables d'environnement ;
- les journaux applicatifs.

Une restauration doit être testée périodiquement.

10.9 Supervision

La version actuelle ne fournit pas d'intégration de supervision. Il est recommandé d'ajouter :

- un collecteur de logs ;
- des métriques applicatives ;
- des alertes sur erreurs 5xx ;
- un suivi de la file d'attente ;
- un suivi des temps de réponse.

10.10 Sécurité de production

- Désactiver l'inscription publique si elle n'est pas souhaitée.
- Restreindre l'accès à `/admin` par le rôle de plateforme.
- Forcer HTTPS et HSTS.
- Configurer un pare-feu.
- Limiter les accès à la base.
- Mettre à jour les dépendances.
- Auditer les comptes de plateforme.
- Définir une politique de rétention du journal.
- Sauvegarder avant chaque migration.

---

11. Bilan et perspectives

11.1 Réalisations

La version livrée apporte :

- une architecture MVC claire avec services métier ;
- un contrôle d'accès centralisé par 14 Policies ;
- un rôle global d'administrateur de plateforme ;
- un journal d'audit transversal ;
- une isolation multi-exploitations à trois niveaux ;
- un module de calendrier cultural idempotent ;
- des fiches de culture ;
- une gestion de stock robuste ;
- un blocage serveur des récoltes trop précoces ;
- un tableau de bord mis en cache ;
- une API REST sécurisée ;
- 93 tests automatisés réussis ;
- une documentation technique et un README à jour.

11.2 Apports techniques

Le projet démontre la maîtrise de :

- Laravel 13 et PHP 8.3 ;
- Eloquent, relations, scopes et traits ;
- Policies et Gates ;
- transactions et verrous pessimistes ;
- événements et listeners ;
- files d'attente et notifications ;
- tests PHPUnit ;
- sécurité applicative ;
- modélisation de base de données ;
- documentation technique.

11.3 Limites

- L'historique Git doit être enrichi.
- La couverture de tests peut être améliorée sur les cas limites.
- La rétention de l'audit n'est pas automatisée.
- L'interface n'a pas de tests end-to-end.
- La supervision de production est absente.
- La documentation utilisateur est synthétique.
- L'API ne couvre qu'une partie du domaine.
- Les contraintes SQL fortes ne s'appliquent qu'à PostgreSQL.

11.4 Perspectives

Court terme

- ajouter des commits structurés et documentés ;
- compléter les tests API ;
- ajouter des tests de concurrence sur le stock ;
- documenter les procédures de sauvegarde et de restauration ;
- ajouter un écran de détail du journal.

Moyen terme

- mettre en place une purge ou un archivage du journal ;
- ajouter l'export CSV/PDF des états ;
- compléter l'API REST ;
- ajouter une documentation OpenAPI ;
- mettre en place une supervision ;
- ajouter des tests navigateur.

Long terme

- application mobile hors ligne ;
- intégration météo ;
- recommandations agronomiques contextualisées ;
- gestion multi-devises ;
- comptabilité analytique ;
- traçabilité réglementaire ;
- tableaux de bord avancés.

11.5 Conclusion

VolyLegume répond aux besoins structurants d'une exploitation maraîchère : suivi de production, sécurité des accès, isolation des données, traçabilité et pilotage. Les renforcements apportés — Policies, audit, administration plateforme, tests — consolident la conformité au cahier des charges et la qualité de la base technique.

Les limites identifiées sont principalement des sujets d'exploitation et d'industrialisation : historique Git, supervision, rétention, tests end-to-end. Elles n'empêchent pas la démonstration fonctionnelle et constituent une feuille de route claire pour les prochaines itérations.

---

12. Annexes

Annexe A — Arborescence simplifiée

```
VolyLegume/
├── app/
│   ├── Enums/
│   ├── Events/
│   ├── Exceptions/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Admin/
│   │   │   └── Api/
│   │   ├── Middleware/
│   │   ├── Requests/
│   │   └── Resources/
│   ├── Jobs/
│   ├── Listeners/
│   ├── Models/
│   │   └── Concerns/
│   ├── Notifications/
│   ├── Policies/
│   │   └── Concerns/
│   ├── Providers/
│   ├── Services/
│   └── Support/
├── database/
│   ├── factories/
│   ├── migrations/
│   └── seeders/
├── resources/
│   └── views/
├── routes/
├── tests/
├── docker/
├── docker-compose.yml
├── composer.json
├── package.json
└── README.md
```

Annexe B — Matrice rôles / actions

Action	Agent	Responsable	Admin exploitation	Admin plateforme
Voir tableau de bord	Oui	Oui	Oui	Oui
Voir parcelles	Oui	Oui	Oui	Oui
Créer/modifier parcelles	Non	Oui	Oui	Oui
Voir cultures	Oui	Oui	Oui	Oui
Créer/modifier cultures	Non	Oui	Oui	Oui
Voir campagnes	Oui	Oui	Oui	Oui
Créer/modifier campagnes	Non	Oui	Oui	Oui
Voir activités	Oui	Oui	Oui	Oui
Créer activités	Oui	Oui	Oui	Oui
Modifier/supprimer activités	Non	Oui	Oui	Oui
Voir stocks	Oui	Oui	Oui	Oui
Entrées/sorties stock	Oui	Oui	Oui	Oui
Ajuster stock	Non	Oui	Oui	Oui
Voir traitements	Oui	Oui	Oui	Oui
Créer traitements	Non	Oui	Oui	Oui
Voir récoltes	Oui	Oui	Oui	Oui

Action	Agent	Responsable	Admin exploitation	Admin plateforme
Créer récoltes	Oui	Oui	Oui	Oui
Supprimer récoltes	Non	Oui	Oui	Oui
Voir finances	Non	Oui	Oui	Oui
Gérer finances	Non	Oui	Oui	Oui
Gérer équipe	Non	Oui	Oui	Oui
Voir journal exploitation	Non	Oui	Oui	Oui
Accéder `/admin`	Non	Non	Non	Oui

Annexe C — Commandes utiles

```
# Installation
composer install
npm install
cp .env.example .env
php artisan key:generate
npm run build

# Base de données
php artisan migrate
php artisan migrate:fresh --seed

# Développement
php artisan serve
npm run dev
php artisan queue:work

# Qualité
php artisan test
vendor/bin/pint

# Docker PostgreSQL
docker compose up -d
```

Annexe D — Glossaire

Terme	Définition
Campagne	Cycle de production d'une culture sur une parcelle
Contexte d'exploitation	Exploitation active pour la requête
Délai avant récolte	Nombre de jours entre un traitement et la récolte autorisée
Intrant	Produit consommé par l'exploitation (semence, engrais, etc.)
Parcelle	Unité physique de production
Policy	Classe Laravel centralisant une décision d'autorisation
Rattachement	Lien entre un utilisateur et une exploitation, avec un rôle
Tenant	Exploitation isolée dans un contexte multi-exploitations
Traitement phytosanitaire	Application d'un produit sur une culture

Annexe E — Références

- Documentation Laravel 13 : https://laravel.com/docs
- Documentation PHPUnit : https://phpunit.de/documentation.html
- Documentation PostgreSQL : https://www.postgresql.org/docs/
- Documentation Sanctum : https://laravel.com/docs/sanctum
- Documentation Tailwind CSS : https://tailwindcss.com/docs
- OWASP Top 10 : https://owasp.org/www-project-top-ten/
